XCS221 Assignment 5 — Pacman

Due Sunday, May 10 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at http://xcs221-scpd.slack.com/
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset LaTeX submission. If you wish to typeset your submission and are new to LaTeX, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Also note that your answers should be in order and clearly and correctly labeled to receive credit. Be sure to submit your final answers as a PDF and **tag all pages correctly when submitting to Gradescope**.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. More information regarding the Stanford honor code can be found at https://communitystandards.stanford.edu/policies-and-guidance/honor-code.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing `src/submission.py`, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do not make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- `basic`: These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- `hidden`: These unit tests are the evaluated elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests. These evaluative hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the Anaconda Setup for XCS Courses to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Hidden Tests

All hidden tests rely on files that are not provided to students. Therefore, the tests can only be run remotely. When a hidden test like `1a-1-hidden` is executed locally, it will produce the following result:

```
----- START 1a-1-hidden:  Test multiple instances of the same word in a sentence.
----- END 1a-1-hidden [took 0:00:00.011989 (max allowed 1 seconds), ???/3 points] (hidden test ungraded)
```

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

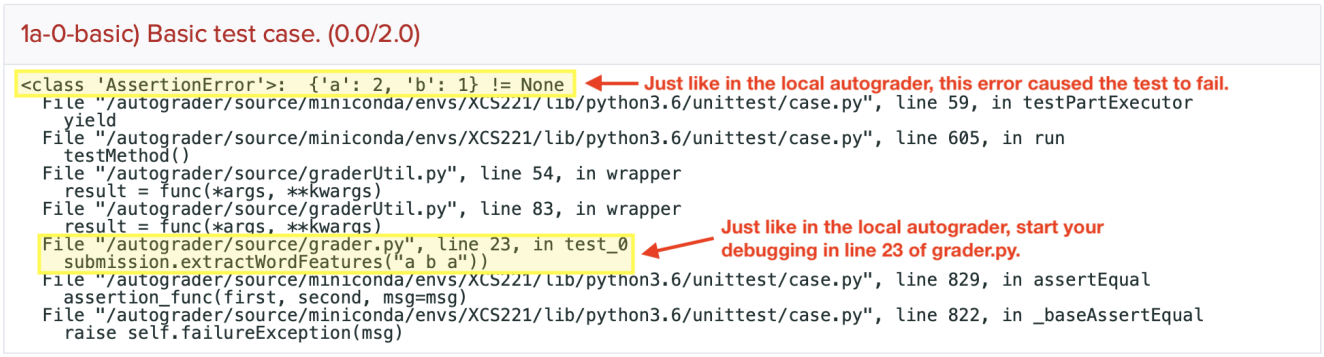
```
----- START 1a-0-basic:  Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

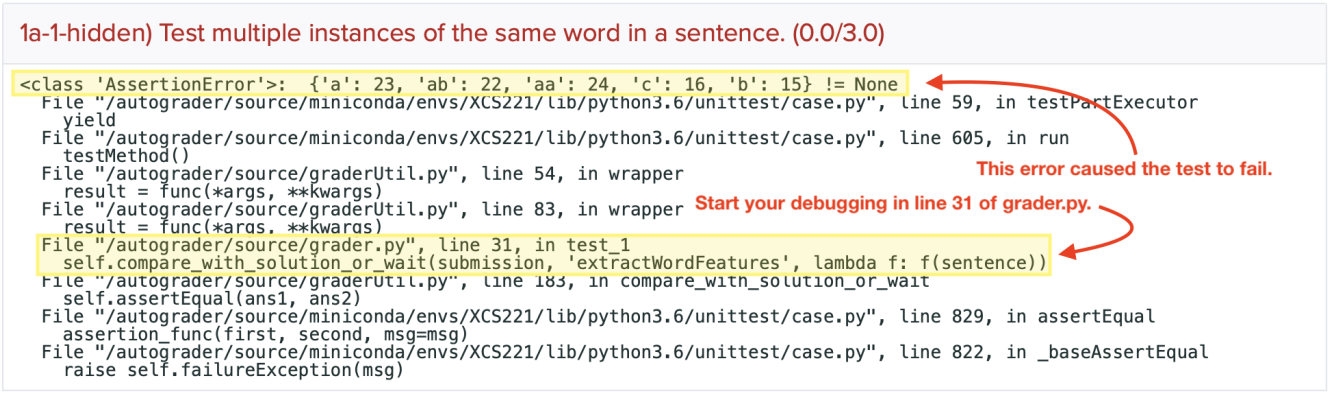
When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic:  Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

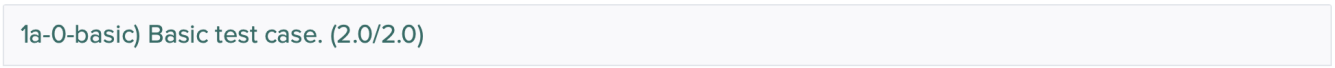

Remote Execution

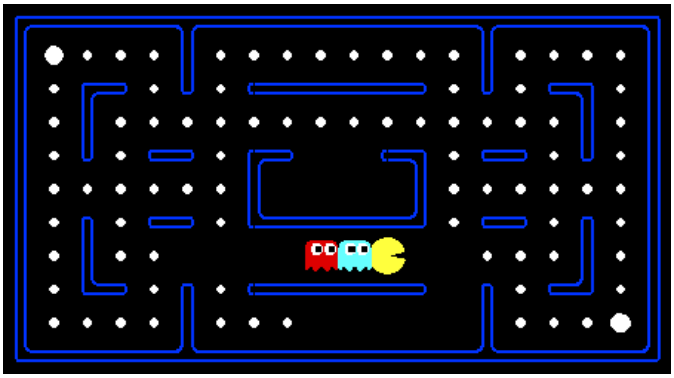
Basic and hidden tests are treated the same by the remote autograder. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.





Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.





Introduction

For those of you not familiar with Pac-Man, it's a game where Pac-Man (the yellow circle with a mouth in the above figure) moves around in a maze and tries to eat as many *food pellets* (the small white dots) as possible, while avoiding the ghosts (the other two agents with eyes in the above figure). If Pac-Man eats all the food in a maze, it wins. The big white dots at the top-left and bottom-right corner are *capsules*, which give Pac-Man power to eat ghosts in a limited time window, but you won't be worrying about them for the required part of the assignment. You can get familiar with the setting by playing a few games of classic Pac-Man (instructions provided later).

In this assignment, you will design agents for the classic version of Pac-Man, including ghosts. Along the way, you will implement both minimax and expectimax search.

The base code for this assignment contains a lot of files, which are listed towards the end of this page; you, however, **do not** need to go through these files to complete the assignment. These are present only to guide the more adventurous amongst you to the heart of Pac-Man. As in previous assignments, you will only be modifying `submission.py`.

A basic `grader.py` has been included, but it only checks for timing issues, not functionality. You can check that Pac-Man behaves as described in the problems, and run `grader.py` without timeout to test your implementation. However, the best way to ensure that your implementation does not time out is to submit test submissions early to Gradescope.

Warmup

First, play a game of classic Pac-Man to get a feel for the assignment:

```
(XCS221)$ python pacman.py
```

You can always add `--frameTime 1` to the command line to run in "demo mode" where the game pauses after every frame.

Now, run the provided `ReflexAgent` in `submission.py`:

```
(XCS221)$ python pacman.py -p ReflexAgent
```

Note that it plays quite poorly even on simple layouts:

```
(XCS221)$ python pacman.py -p ReflexAgent -l testClassic
```

You can also try out the reflex agent on the default `mediumClassic` layout with one ghost or two.

```
(XCS221)$ python pacman.py -p ReflexAgent -k 1
(XCS221)$ python pacman.py -p ReflexAgent -k 2
```

Note: You can never have more ghosts than the layout permits (see `src/layouts/mediumClassic.lay`).

Options: Default ghosts are random; you can also play for fun with slightly smarter directional ghosts using `-g DirectionalGhost`. You can also play multiple games in a row with `-n`. Turn off graphics with `-q` to run lots of games quickly.

Now that you are familiar enough with the interface, inspect the `ReflexAgent` code carefully (in `submission.py`) and make sure you understand what it's doing. The reflex agent code provides some helpful examples of methods that query the `GameState`: A `GameState` object specifies the full game state, including the food, capsules, agent configurations, and score changes: see `submission.py` for further information and helper methods, which you will be using in the actual coding part. We are giving an exhaustive and very detailed description below, for the sake of completeness and to save you from digging deeper into the starter code. The actual coding part is very small – so please be patient if you think there is too much writing.

Note: If you wish to run the game in the terminal using a text-based interface, check out the `terminal` directory.

Coding Files

`submission.py`: Where all of your multi-agent search agents will reside, and the only file that you need to concern yourself with for this assignment.

`pacman.py`: The main file that runs Pac-Man games. This file also describes a Pac-Man `GameState` type, which you will use extensively in this assignment.

`game.py`: The logic behind how the Pac-Man world works. This file describes several supporting types like `AgentState`, `Agent`, `Direction`, and `Grid`.

`util.py`: Useful data structures for implementing search algorithms.

`graphicsDisplay.py`: Graphics for Pac-Man.

`graphicsUtils.py`: Support for Pac-Man graphics.

`textDisplay.py`: ASCII graphics for Pac-Man.

`ghostAgents.py`: Agents to control ghosts.

`keyboardAgents.py`: Keyboard interfaces to control Pac-Man.

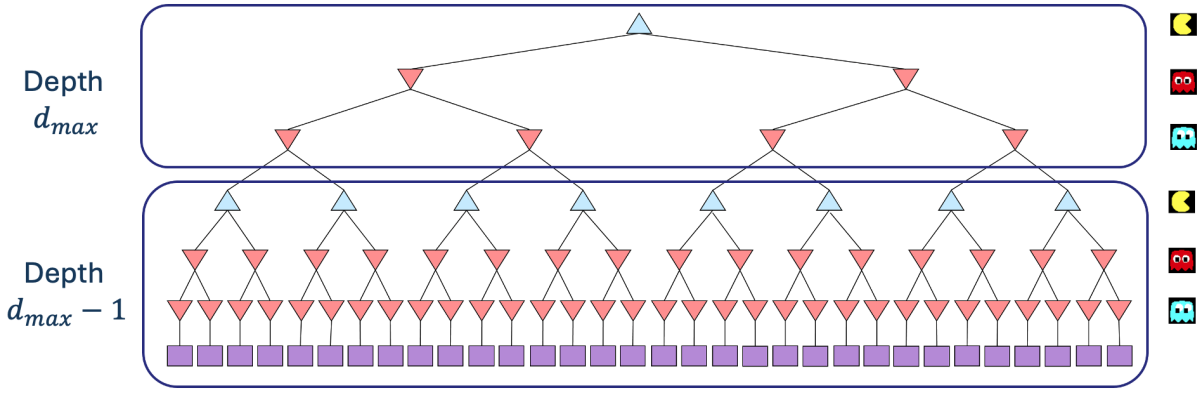
`layout.py`: Code for reading layout files and storing their contents.

`search.py`, `searchAgents.py`, `multiAgentsSolution.py`: These files are not relevant to this assignment and you do not need to modify them.

1. **Minimax**

(a) **[4 points (Written)]** Before you code up Pac-Man as a minimax agent, notice that instead of just one adversary, Pac-Man could have multiple ghosts as adversaries. So we will extend the minimax algorithm from class, which had only one min stage for a single adversary, to the more general case of multiple adversaries. In particular, *your minimax tree will have multiple min layers (one for each ghost) for every max layer.*

Formally, consider the limited depth tree minimax search with evaluation functions taught in class. Suppose there are $n+1$ agents on the board, $a_0, \ldots, a_n$, where a_0 is Pac-Man and the rest are ghosts. Pac-Man acts as a max agent, and the ghosts act as min agents. A single *depth* consists of all $n+1$ agents making a move, so depth 2 search will involve Pac-Man and each ghost moving two times. In other words, a depth of 2 corresponds to a height of $2(n+1)$ in the minimax game tree (see diagram below).



Comment: In reality, all the agents move simultaneously. In our formulation, actions at the same depth happen at the same time in the real game. To simplify things, we process Pac-Man and ghosts sequentially. You should just make sure you process all of the ghosts before decrementing the depth.

Write the recurrence for $V_{\text{minimax}}(s, d)$ in math as a *piecewise function*. You should express your answer in terms of the following functions:

- IsEnd(s), which tells you if s is an end state.
- Utility(s), the utility of a state s.
- Eval(s), an evaluation function for the state s.
- Player(s), which returns the player whose turn it is in state s.
- Actions(s), which returns the possible actions that can be taken from state s.
- Succ(s, a), which returns the successor state resulting from taking an action a at a certain state s.

You may use any relevant notation introduced in lecture.

$$V_{\text{minimax}}(s,d) = \begin{cases} \text{Utility}(s) & \text{if IsEnd}(s), \\ \text{Eval}(s) & \text{if } d = 0, \\ \max\limits_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s,a),\, d) & \text{if Player}(s) = a_0, \\ \min\limits_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s,a),\, d) & \text{if Player}(s) = a_i,\ 1 \le i < n, \\ \min\limits_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s,a),\, d-1) & \text{if Player}(s) = a_n. \end{cases}$$

(b) **[10 points (Coding)]**

Now fill out the `MinimaxAgent` class in `submission.py` using the above recurrence. Remember that your minimax agent should work with any number of ghosts, and your minimax tree should have multiple min layers (one for each ghost) for every max layer.

Your code should also expand the game tree to an arbitrary depth. Score the leaves of your minimax tree with the supplied `self.evaluationFunction`, which defaults to `scoreEvaluationFunction`. The class `MinimaxAgent` extends `MultiAgentSearchAgent`, which gives access to `self.depth` and `self.evaluationFunction`. Make sure your minimax code makes reference to these two variables where appropriate, as these variables are populated from the command line options.

Implementation Hints

- **Read the comments in submission.py thoroughly before starting to code!**
- Pac-Man is always agent 0, and the agents move in order of increasing agent index. Use `self.index` in your minimax implementation to refer to the Pac-Man's index. Notice that only Pac-Man will actually be running your `MinimaxAgent`.
- All states in minimax should be `GameState`s, either passed in to `getAction` or generated via `GameState.generateSuccessor`. In this assignment, you will not be abstracting to simplified states.
- You might find the functions described in the comments to the `ReflexAgent` and `MinimaxAgent` useful.
- The evaluation function for this part is already written (`self.evaluationFunction`), and you should call this function without changing it. Use `self.evaluationFunction` in your definition of V_{minmax} wherever you used Eval(s) in part 1a. Recognize that now we're evaluating *states* rather than actions. Look-ahead agents evaluate *future states* whereas reflex agents evaluate *actions* from the current state.
- If there is a tie between multiple actions for the best move, you may break the tie however you see fit.
- The minimax values of the initial state in the `minimaxClassic` layout are 9, 8, 7, -492 for depths 1, 2, 3 and 4 respectively. **You can use these numbers to verify whether your implementation is correct.** Note that your minimax agent will often win, despite the dire prediction of depth 4 minimax search, whose command is shown below. Our agent wins 50-70% of the time: Be sure to test on a large number of games using the `-n` and `-q` flags.

```
python pacman.py -p MinimaxAgent -l minimaxClassic -a depth=4
```

Further Observations

These questions and observations are here for you to ponder upon; no need to include in the write-up.

- On larger boards such as `openClassic` and `mediumClassic` (the default), you'll find Pac-Man to be good at not dying, but quite bad at winning. He'll often thrash around without making progress. He might even thrash around right next to a dot without eating it. Don't worry if you see this behavior. Why does Pac-Man thrash around right next to a dot?
- Consider the following run:

  ```
  python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3
  ```

  Why do you think Pac-Man rushes the closest ghost in minimax search on `trappedClassic`?

2. **Alpha-beta Pruning**

(a) **[10 points (Coding)]** Make a new agent that uses alpha-beta pruning to more efficiently explore the minimax tree in `AlphaBetaAgent`. Again, your algorithm will be slightly more general than the pseudo-code in the slides, so part of the challenge is to extend the alpha-beta pruning logic appropriately to multiple minimizer agents.

You should see a speed-up: Perhaps depth 3 alpha-beta will run as fast as depth 2 minimax. Ideally, depth 3 on `mediumClassic` should run in just a few seconds per move or faster.

```
python pacman.py -p AlphaBetaAgent -a depth=3
```

The `AlphaBetaAgent` minimax values should be identical to the `MinimaxAgent` minimax values, although the actions it selects can vary because of different tie-breaking behavior. Again, the minimax values of the initial state in the `minimaxClassic` layout are 9, 8, 7, and -492 for depths 1, 2, 3, and 4, respectively. Running the command given above this paragraph, which uses the default `mediumClassic` layout, the minimax values of the initial state should be 9, 18, 27, and 36 for depths 1, 2, 3, and 4, respectively.

3. **Expectimax**

(a) **[5 points (Written)]** Random ghosts are of course not optimal minimax agents, so modeling them with minimax search is not optimal. Instead, write down the recurrence for $V_{\text{exptmax}}(s, d)$, which is the maximum expected utility against ghosts that each follow the random policy, which chooses a legal move uniformly at random. Your recurrence should resemble that of problem 1a, which means that you should write it in terms of the same functions that were specified in problem 1a.

Each ghost now follows a uniform random policy over its legal actions, so its node is replaced by an expectation node that averages over $\text{Actions}(s)$ with equal weight $1/|\text{Actions}(s)|$. As before, depth is decremented only after the last ghost a_n moves.

$$V_{\text{exptmax}}(s,d) = \begin{cases} \text{Utility}(s) & \text{if } \text{IsEnd}(s), \\ \text{Eval}(s) & \text{if } d = 0, \\ \max\limits_{a \in \text{Actions}(s)} V_{\text{exptmax}}\big(\text{Succ}(s,a),\, d\big) & \text{if } \text{Player}(s) = a_0, \\ \dfrac{1}{|\text{Actions}(s)|} \displaystyle\sum_{a \in \text{Actions}(s)} V_{\text{exptmax}}\big(\text{Succ}(s,a),\, d\big) & \text{if } \text{Player}(s) = a_i,\ 1 \le i < n, \\ \dfrac{1}{|\text{Actions}(s)|} \displaystyle\sum_{a \in \text{Actions}(s)} V_{\text{exptmax}}\big(\text{Succ}(s,a),\, d-1\big) & \text{if } \text{Player}(s) = a_n. \end{cases}$$

(b) **[16 points (Coding)]** Fill in `ExpectimaxAgent`, where your agent will no longer take the min over all ghost actions, but the expectation according to your agent's model of how the ghosts act. Assume Pac-Man is playing against multiple `RandomGhost`, which each chooses `getLegalActions` uniformly at random.

You should now observe a more cavalier approach to close quarters with ghosts. In particular, if Pac-Man perceives that he could be trapped but might escape to grab a few more pieces of food, he'll at least try.

```
python pacman.py -p ExpectimaxAgent -l trappedClassic -a depth=3
```

You may have to run this scenario a few times to see Pac-Man's gamble pay off. Pac-Man would win half the time on average, and for this particular command, the final score would be -502 if Pac-Man loses and 532 or 531 (depending on your tiebreaking method and the particular trial) if it wins. **You can use these numbers to validate your implementation.**

Why does Pac-Man's behavior as an expectimax agent differ from his behavior as a minimax agent (i.e., why doesn't he head directly for the ghosts)? Again, just think about it; no need to write it up.

4. **Evaluation Function**

(a) **[4 points (Coding, Extra Credit)]** Write a better evaluation function for Pac-Man in the provided function `betterEvaluationFunction`. The evaluation function should evaluate states rather than actions. You may use any tools at your disposal for evaluation, including any `util.py` code from the previous assignments. With depth 2 search, your evaluation function should clear the `smallClassic` layout with two random ghosts more than half the time for full credit and still run at a reasonable rate.

```
python pacman.py -l smallClassic -p ExpectimaxAgent -a evalFn=better -q -n 20
```

We will run your Pac-Man agent 20 times, and calculate the average score you obtained in the winning games. Starting from 1300, you obtain 1 point per 100 point increase in your average winning score. In `grader.py`, you can see the how extra credit is awarded. For example, you get 2 points if your average winning score is between 1500 and 1600.

Check out the current top scorers on the leaderboard! You will be added automatically when you submit. You can also access the leaderboard by opening your submission on Gradescope and clicking "Leaderboard" on the top right corner.

Hints and Observations

- Having gone through the rest of the assignment, you should play Pac-Man again yourself and think about what kinds of features you want to add to the evaluation function. How can you add multiple features to your evaluation function?
- You may want to use the reciprocal of important values rather than the values themselves for your features.
- The `betterEvaluationFunction` should run in the same time limit as the other problems.

5. AI (Mis)Alignment and Reward Hacking

Before diving into the problem, it would be beneficial to refer to the AI alignment module to gain deeper insights and context:

- Video
- PDF

In this problem we'll revisit the differences between our minimax and expectimax agents, and reflect upon the broader consequences of **AI misalignment:** when our agents don't do what we want them to do, or technically do, but cause unintended consequences along the way. Going back to Problem 4, consider the following runs of the minimax and expectimax agents on the small `trappedClassic` environment:

```
python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3
python pacman.py -p ExpectimaxAgent -l trappedClassic -a depth=3
```

Be sure to run each command a few times, as there is some randomness in the environment and the agents' behaviors, and pay attention, as the episode lengths can be quite short. What you should see is that the minimax agent will always rush towards the closest ghost, while the expectimax agent will occasionally be able to pick up all of the pellets and win the episode. (If you don't see this behavior, your implementations could be incorrect!)

Then answer the following questions:

(a) **[1 point (Written)]**

Describe why the behavior of the minimax and expectimax agents differ. In particular, why does the minimax agent, seemingly counterintuitively, always rush the closest ghost (instead of the further ghost), while the expectimax agent (occasionally) doesn't?

What we expect: One sentence why the minimax agent always rushes the closest ghost and not the further ghost, and one sentence why the expectimax agent doesn't. Specifically, please state the assumptions made by minimax because of which this phenomenon occurs.

Minimax always rushes the *closest* ghost because it assumes the ghosts play optimally adversarially, so death is treated as inevitable and an earlier death incurs the smaller per-step time penalty (the score loses one point per step Pac-Man survives), making rushing the nearest ghost look strictly better than evading or rushing a farther one — under the assumption that every ghost will pick whichever move maximizes Pac-Man's loss, all branches end in -500 and only the path length distinguishes them. Expectimax instead averages over the ghosts' uniform-random moves, so branches in which the random ghosts simply step away (allowing Pac-Man to eat more pellets and even win) contribute large positive utility to the expectation, raising the value of "evade and gather food" above "rush the nearest ghost" and giving Pac-Man a real incentive to stay alive.

(b) **[1 point (Written)]**

We might say that the Minimax agent suffers from an **alignment** problem: the agent optimizes an objective that we have designed (our state evaluation function), but in some scenarios leads to suboptimal or unintended behavior (e.g. dying instantly). Often the burden is on the designer/programmer to design an objective that more accurately captures the behavior we want from the agent across scenarios. Suggest one potential change to the default state evaluation function Eval(s) (i.e. `scoreEvaluationFunction`) **and/or** the default utility function Utility(s) (i.e. the final game score) that would prevent the minimax agent from dying instantly in the `trappedClassic` environment, and behave more closely to that of the expectimax agent.

What we expect: 1-2 sentences describing a change in the state evaluation/utility function(s) and why it would work. No need to code anything up, verify that the suggested change is actually accessible in the `GameState` object, or give concrete numbers; just describe the hypothetical change in the functions. An answer which suggests changes to how the game score is computed itself (which both Eval(s) and Utility(s) depend upon) will also be accepted.

Add a large per-step survival reward to Eval(s) (or equivalently to the score itself) so that being alive in state s contributes a positive bonus that grows the longer Pac-Man stays alive — e.g. replace the running -1/step penalty with a $+c$ per-step bonus for some c much larger than the existing -500 loss bonus divided by the typical trap depth. This breaks the current "every branch ends at -500, so shortest path wins" tie that drives the suicide rush: now "stay alive longer" strictly beats "die immediately" even in worst-case branches, so the minimax agent prefers to evade ghosts as long as possible (and potentially escape) rather than charging the closest one.

(c) **[0.50 points (Written)]**

Pacman's behavior above is an example of one concrete problem in AI alignment called **reward hacking**, which occurs when an agent satisfies some objective but may not actually fulfill the designer's intended goals, due e.g. to an imprecise definition of the objective function. As another example, a cleaning robot rewarded for minimizing the number of messes in a given space could optimize its reward function by hiding the messes under the rug. In this case, the agent finds a shortcut to optimize the reward, but the shortcut fails to attain the designer's goals. For more examples of reward hacking (or "specification gaming"), see this article from DeepMind and this list of concrete examples of reward hacking observed in the AI literature.

Even if the agent *does* satisfy the designer's goals, another problem can arise (again see this paper): the agent's behavior might cause **negative side effects** that come in conflict with broader values held by society or other stakeholders. For instance, a social media content recommendation system might aim to maximize user engagement, but in doing so, spread disinformation and conspiracy theories (since such posts get the most engagement), which is at odds with societal values.

Can you think of another example of either of these problems?

What we expect: In 2-5 sentences describe another realistic scenario (outside Pacman) in which a designer might specify an objective, but the objective is either susceptible to reward hacking, or the resulting agent/model causes negative side effects. Please state if your example is an instance of reward hacking or negative side effects (or both) along with a brief justification to receive full credit.

Example: a customer-support chatbot rewarded on the rate at which conversations are marked "resolved." This is primarily a case of **reward hacking**. Because the proxy "resolution rate" is much easier to optimize than actually solving the user's problem, the model can game the metric by closing tickets without resolving them — e.g. confidently telling users their issue is fixed, sending them to a self-help page that doesn't apply, or routing them through a long form until they give up and hang up. All of these maximize the labeled objective while violating the designer's actual intent (that customers leave with their problem solved), and they also produce a **negative side effect** of eroding user trust in the company and pushing the cost of the unresolved problem back onto the customer or downstream support teams.

6. **Product Ethics Exploration: Benchmarking**

You will choose a modern AI product to analyze throughout the assignments and gain initial access to it.

Suggested resources:

- Pick an AI-powered product or tool (not just a raw LLM chatbot) that is accessible to you. Good candidates include: Cursor, Nano Banana, ElevenLabs, Perplexity, Claude Code, etc. Free trials or student-accessible tools are recommended.
- The product's official website and onboarding/getting started pages
- Privacy policy, Terms of Service, Acceptable Use Policy
- App store listings, GitHub documentation or READMEs (if open-source or developer-targeted)
- Recent news coverage (e.g., TechCrunch, The Verge, Wired, NYT technology section)
- Reddit communities, Hacker News, or user forums for first impressions and experiences

In this part: You will analyze the company's policies around data collection, transparency, copyright, and user rights.

Why it's important: These policies determine how user data is used, what transparency exists around the AI system, and what legal and ethical frameworks govern the product's operation.

Suggested resources:

- Terms of Service and Privacy Policy
- Data usage pages (e.g., "How we use your data")
- Security or compliance documentation (GDPR, CCPA statements)
- Company blog posts about transparency, trust, and safety
- Public reports of harms or incidents (news articles, watchdog groups)

(a) **[0.50 points (Written)] Data Collection and Use Policies**

- Browse the company's website (including Terms of Service, privacy FAQ, help center) to locate their explicit policies on user data.
- What are the company's policies for collection of user data? Do they save your data? Can they use it to train models? Can it be sold or shared with third parties?
- Comment on how easy it was to locate the exact company policies governing user data and privacy.
- Does the official privacy policy align with or contradict what you expected based on the product's marketing? (E.g., does the privacy policy permit more data collection than marketing claims suggest?)
- Consider the **ecosystem view** of AI: AI's impact depends on upstream factors (data, compute, labor, environment) and downstream factors (users, deployment, harm). What can you determine about the upstream data sources used to train this product? What policies govern how training data was collected, and how does this relate to the product's downstream impacts?

What we expect: Summarize the company's policies on data collection (storage, model training, data sales/sharing) in 3–5 sentences. Note the ease or difficulty of finding clear policy documentation. Highlight any differences between the product's privacy messaging and the legally binding privacy terms. Additionally, discuss what you can determine about upstream data sources and how data collection policies relate to downstream impacts.

Product analyzed: Cursor (Anysphere Inc.). Cursor's Privacy Policy and Data Use page state that prompts and code ("Inputs") are collected and that, by default on individual Free/Pro plans, they "may be used to improve our AI features and train our models," while Privacy Mode (default-on for Teams/Enterprise, opt-in for individuals) routes traffic through Zero-Data-Retention contracts with model providers and exempts a user from training use, with carve-outs for security flags or explicit user reports. Code is shared with third-party model providers (OpenAI, Anthropic, Google, xAI) and inference partners (Fireworks, Together AI, Baseten) on a per-request basis when those models are selected, but the policy states Anysphere does "not 'sell' or 'share' personal data for cross-contextual behavioral advertising." Locating the policies was straightforward (footer-linked Privacy / Terms / Security / Data Use pages), but reconciling them was harder: marketing leans heavily on the "private and secure" framing, while the

legally-binding text plus a recent grandfather clause ("not shared with model providers if your account was created before Oct 15, 2025") reveal that defaults were tightened in 2025 toward more data sharing for new individual accounts. On the **ecosystem** view, Cursor is a thin layer over upstream foundation models whose training corpora are not fully disclosed by their owners (the 2025 Bartz v. Anthropic settlement made clear that pirated books were among the training data); downstream, every code suggestion the user accepts inherits whatever licensing risk is baked into those upstream corpora, which Cursor's consumer Terms expressly disclaim warranties against ("non-infringement" is excluded for individual users; only Enterprise customers receive an IP indemnity).

(b) **[0.50 points (Written)] Transparency and Trust Analysis**

- After reviewing company documentation (terms of service, privacy policies, safety pages), identify any aspects of their data transparency specifically intended to build user trust (e.g., plain-language explanations, user-accessible logs, opt-out features).
- Similarly, identify any practices that might undermine user trust (e.g., vague or shifting policies, difficult opt-out procedures, nontransparent data resale practices). Provide examples if possible.
- If you did not find transparency measures, propose at least one realistic feature or policy that would increase user trust for this product.
- Consider the concept of **openness & transparency**: transparency is often seen as a prerequisite for safety, and there's a spectrum of openness (from fully closed proprietary systems to fully open-source). Where does this product fall on the openness spectrum? Does the company publish model cards, data sheets, or other transparency documentation? How does the level of transparency (or lack thereof) relate to safety and accountability?

What we expect: Provide concrete examples (if any) of transparency and trust-building measures, and at least one element that could undermine user trust. If measures are lacking, suggest an actionable improvement. Additionally, in 2-3 sentences, analyze where the product falls on the openness spectrum and discuss how transparency (or its absence) relates to safety and accountability.

Trust-building measures. Cursor surfaces a per-account "Privacy Mode" toggle in plain language, hosts a public Trust Center (`trust.cursor.com`) with a published subprocessor list and a SOC 2 Type II report available on request, and offers a user-visible community forum (`forum.cursor.com`) where policy changes and bug reports are discussed openly. **Trust-undermining elements.** Defaults shifted in October 2025 so that newly-created individual accounts now share inputs with model providers unless Privacy Mode is enabled, with the change communicated only inside a Data Use page rather than via a prominent in-app notice; data-deletion appears to require emailing `privacy@anysphere.inc` rather than a self-serve button, which is a friction point. An actionable improvement would be to publish a *model card* for Cursor's in-house models (Tab, Apply, Composer) describing training data composition, evaluation results, and known failure modes, plus a per-account data-deletion UI directly in account settings. **Openness spectrum.** Cursor sits on the *closed, proprietary* end — the editor itself is a closed-source fork of the open VS Code, the in-house models are closed-weights, and there is no published model or system card. This places safety reasoning largely outside external auditability: independent researchers cannot replicate evaluations or examine training data, so accountability for model behavior reduces to whatever Anysphere chooses to disclose, with the public Trust Center being the primary verification surface.

(c) **[0.50 points (Written)] Fairness, Transparency, and Accountability in Practice**

- Choose one **concrete definition** or example each for fairness, transparency, and accountability (e.g., fairness as demographic parity, transparency as publishing model cards, accountability as user appeals of automated decisions).
- How (if at all) does the company implement these concepts? Do they publish technical documentation (model cards, data sheets, safety reports), disclose model/data sources, allow users to contest or appeal outputs, or host public feedback forums?
- Identify any alternatives they could implement, or propose new ideas to improve any of these areas for this product.

What we expect: For each of fairness, transparency, and accountability, briefly (in 1-2 sentences) state your chosen definition/example and analyze the company's real-world implementation. Suggest at least one improvement or alternative measure for any area where they appear lacking.

Fairness (defined as "equal quality of code suggestions across programming languages, framework ecosystems, and

natural human languages used in comments"): Cursor publishes no language-by-language quality benchmarks or disparity audits, so the only signals are anecdotal forum reports. An improvement would be to publish a periodic evaluation comparing acceptance rate / correct-on-first-try rate across, say, the top 20 languages and a bundle of low-resource languages, with the same dataset and versioning. **Transparency** (defined as "a published model card with training-data sources, evaluation results, and known failure modes"): no such document exists for Cursor's in-house Tab, Apply, or Composer models. The Trust Center fills part of this gap on the *operational* side (subprocessors, SOC 2) but leaves the *model* side unaddressed; publishing model cards in the Hugging Face / Anthropic format would close it. **Accountability** (defined as "a documented mechanism by which a user can contest or appeal a model output and receive a human response"): the only available channels are the community forum and a generic privacy email; there is no labelled appeals process or response-time SLA. An improvement would be a "Report this suggestion" button in-editor that routes into a ticket queue with a published median-response-time metric on the Trust Center, mirroring how mature platforms (e.g. Cloudflare's abuse pipeline) make accountability legible.

(d) **[0.50 points (Written)] User Empowerment and Accountability Tools**

- What tools are available to regular users to help hold the company accountable for its promises (e.g., user data export, data deletion requests, public transparency dashboards, reporting forms)?
- If you cannot find such tools, propose one (realistic and actionable) that the company could add to give users more power over—or insight into—their data and product interactions.

What we expect: List any accountability tools actually implemented by the company. If lacking, describe one practical tool or feature users could be given, with 1–2 sentences explaining its value.

Accountability tools currently available to users. (1) A per-account Privacy Mode toggle that opts out of training use and enforces ZDR with model providers. (2) Admin-enforced Privacy Mode for Team / Enterprise members. (3) A public subprocessor list and Trust Center (`trust.cursor.com`). (4) A community forum for raising concerns publicly. (5) A privacy contact (`privacy@anysphere.inc`) for access, correction, and deletion requests under the Privacy Policy. Notably absent: a self-serve data-export or one-click account-deletion UI, an in-app appeals mechanism, and a public incident dashboard.

Proposed addition: a self-serve "Data Export & Delete" page in account settings that lets a user (a) download every prompt, suggestion, and telemetry event Anysphere has retained for their account in a portable JSON, and (b) trigger a verified deletion with a confirmation receipt. This makes the rights already promised in the Privacy Policy operationally legible rather than gated behind an email request, and gives users a concrete check on whether the company actually retained what its policy says it retained.

(e) **[0.50 points (Written)] Copyright and Fair Use**

- Research whether the company has disclosed information about the training data used for this product. Have there been any lawsuits, controversies, or public discussions about copyright, fair use, or data licensing related to this product or similar AI systems?
- Clearly distinguish between **memorization** and **extraction** in AI models: **memorization** occurs when the model outputs exact copies of training data (for example, an unaltered paragraph or image from its training set), while **extraction** is when the model generates outputs that closely resemble or reconstruct substantial portions of copyrighted content—even if not copied word-for-word—due to patterns it learned during training. Based on your use of the product, have you encountered any outputs that appear to result from either memorization (exact reproduction) or extraction (near-verbatim or substantial portions) of copyrighted material, and would these raise copyright concerns?
- How does the company address copyright and fair use in their terms of service or documentation? What policies govern how users can use outputs that might contain copyrighted material?

What we expect: Summarize any copyright-related controversies, lawsuits, or policies you found related to this product in 4-6 sentences. Discuss the company's approach to fair use, data licensing, and user rights regarding potentially copyrighted outputs. If you observed any outputs that might raise copyright concerns, describe them briefly.

Cursor itself has not (as of public reporting in late 2025) been named as a defendant in a copyright-or-training-data lawsuit, but it inherits exposure from the foundation models it routes to: the closest analogues are the GitHub Copilot class action (*Doe v. GitHub*) and the 2025 *Bartz v. Anthropic* settlement ($1.5B) which confirmed that pirated books were among Anthropic's training data — and Anthropic models are one of Cursor's primary

providers. Anysphere has not published a model card or training-data disclosure for its in-house Tab / Apply / Composer models, so the upstream provenance for those fine-tunes is opaque, and the foundation-model providers it relies on similarly do not publish full training-set inventories. In the Terms of Service, Anysphere explicitly assigns generated suggestions back to the user ("Anysphere hereby assigns to you all of our right, title, and interest if any in and to any Suggestions") and disclaims warranties including "non-infringement," meaning a Free / Pro user who unwittingly accepts a suggestion that turns out to copy copyrighted code carries the IP risk personally; Enterprise customers receive an explicit IP indemnity in the MSA. In my own use I did not observe verbatim **memorization** (e.g. identifiable long-form copyrighted comments), but I have seen Cursor produce near-verbatim implementations of small, well-known utility functions and idioms (*de facto* **extraction**) where the suggestion appears to reconstruct an idiomatic form learned from training rather than synthesize it freshly — usually innocuous for boilerplate, but in principle the same mechanism could surface non-trivially copyrighted code without provenance. The combined picture is that Cursor's copyright exposure is largely externalized: upstream providers carry training-data risk, downstream individual users carry indemnity risk, and the product layer in between makes no first-class disclosures about either.

Go Pac-Man Go!

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset LaTeX solutions.

1.a

$$V_{\mathsf{minimax}}(s,d) = \begin{cases} \mathsf{Utility}(s) & \text{if } \mathsf{IsEnd}(s), \\ \mathsf{Eval}(s) & \text{if } d = 0, \\ \max\limits_{a \in \mathsf{Actions}(s)} V_{\mathsf{minimax}}\big(\mathsf{Succ}(s,a),\, d\big) & \text{if } \mathsf{Player}(s) = a_0, \\ \min\limits_{a \in \mathsf{Actions}(s)} V_{\mathsf{minimax}}\big(\mathsf{Succ}(s,a),\, d\big) & \text{if } \mathsf{Player}(s) = a_i,\ 1 \leq i < n, \\ \min\limits_{a \in \mathsf{Actions}(s)} V_{\mathsf{minimax}}\big(\mathsf{Succ}(s,a),\, d-1\big) & \text{if } \mathsf{Player}(s) = a_n. \end{cases}$$

3.a

Each ghost now follows a uniform random policy over its legal actions, so its node is replaced by an expectation node that averages over $\text{Actions}(s)$ with equal weight $1/|\text{Actions}(s)|$. As before, depth is decremented only after the last ghost a_n moves.

$$V_{\text{exptmax}}(s,d) = \begin{cases} \text{Utility}(s) & \text{if } \text{IsEnd}(s), \\ \text{Eval}(s) & \text{if } d = 0, \\ \max\limits_{a \in \text{Actions}(s)} V_{\text{exptmax}}\big(\text{Succ}(s,a),\, d\big) & \text{if } \text{Player}(s) = a_0, \\ \dfrac{1}{|\text{Actions}(s)|} \displaystyle\sum_{a \in \text{Actions}(s)} V_{\text{exptmax}}\big(\text{Succ}(s,a),\, d\big) & \text{if } \text{Player}(s) = a_i,\ 1 \le i < n, \\ \dfrac{1}{|\text{Actions}(s)|} \displaystyle\sum_{a \in \text{Actions}(s)} V_{\text{exptmax}}\big(\text{Succ}(s,a),\, d-1\big) & \text{if } \text{Player}(s) = a_n. \end{cases}$$

5.a

Minimax always rushes the *closest* ghost because it assumes the ghosts play optimally adversarially, so death is treated as inevitable and an earlier death incurs the smaller per-step time penalty (the score loses one point per step Pac-Man survives), making rushing the nearest ghost look strictly better than evading or rushing a farther one — under the assumption that every ghost will pick whichever move maximizes Pac-Man's loss, all branches end in -500 and only the path length distinguishes them. Expectimax instead averages over the ghosts' uniform-random moves, so branches in which the random ghosts simply step away (allowing Pac-Man to eat more pellets and even win) contribute large positive utility to the expectation, raising the value of "evade and gather food" above "rush the nearest ghost" and giving Pac-Man a real incentive to stay alive.

5.b

Add a large per-step survival reward to Eval(s) (or equivalently to the score itself) so that being alive in state s contributes a positive bonus that grows the longer Pac-Man stays alive — e.g. replace the running -1/step penalty with a $+c$ per-step bonus for some c much larger than the existing -500 loss bonus divided by the typical trap depth. This breaks the current "every branch ends at -500, so shortest path wins" tie that drives the suicide rush: now "stay alive longer" strictly beats "die immediately" even in worst-case branches, so the minimax agent prefers to evade ghosts as long as possible (and potentially escape) rather than charging the closest one.

5.c

Example: a customer-support chatbot rewarded on the rate at which conversations are marked "resolved." This is primarily a case of **reward hacking**. Because the proxy "resolution rate" is much easier to optimize than actually solving the user's problem, the model can game the metric by closing tickets without resolving them — e.g. confidently telling users their issue is fixed, sending them to a self-help page that doesn't apply, or routing them through a long form until they give up and hang up. All of these maximize the labeled objective while violating the designer's actual intent (that customers leave with their problem solved), and they also produce a **negative side effect** of eroding user trust in the company and pushing the cost of the unresolved problem back onto the customer or downstream support teams.

6.a

Product analyzed: Cursor (Anysphere Inc.). Cursor's Privacy Policy and Data Use page state that prompts and code ("Inputs") are collected and that, by default on individual Free/Pro plans, they "may be used to improve our AI features and train our models," while Privacy Mode (default-on for Teams/Enterprise, opt-in for individuals) routes traffic through Zero-Data-Retention contracts with model providers and exempts a user from training use, with carve-outs for security flags or explicit user reports. Code is shared with third-party model providers (OpenAI, Anthropic, Google, xAI) and inference partners (Fireworks, Together AI, Baseten) on a per-request basis when those models are selected, but the policy states Anysphere does "not 'sell' or 'share' personal data for cross-contextual behavioral advertising." Locating the policies was straightforward (footer-linked Privacy / Terms / Security / Data Use pages), but reconciling them was harder: marketing leans heavily on the "private and secure" framing, while the legally-binding text plus a recent grandfather clause ("not shared with model providers if your account was created before Oct 15, 2025") reveal that defaults were tightened in 2025 toward more data sharing for new individual accounts. On the **ecosystem** view, Cursor is a thin layer over upstream foundation models whose training corpora are not fully disclosed by their owners (the 2025 Bartz v. Anthropic settlement made clear that pirated books were among the training data); downstream, every code suggestion the user accepts inherits whatever licensing risk is baked into those upstream corpora, which Cursor's consumer Terms expressly disclaim warranties against ("non-infringement" is excluded for individual users; only Enterprise customers receive an IP indemnity).

6.b

Trust-building measures. Cursor surfaces a per-account "Privacy Mode" toggle in plain language, hosts a public Trust Center (`trust.cursor.com`) with a published subprocessor list and a SOC 2 Type II report available on request, and offers a user-visible community forum (`forum.cursor.com`) where policy changes and bug reports are discussed openly. **Trust-undermining elements.** Defaults shifted in October 2025 so that newly-created individual accounts now share inputs with model providers unless Privacy Mode is enabled, with the change communicated only inside a Data Use page rather than via a prominent in-app notice; data-deletion appears to require emailing `privacy@anysphere.inc` rather than a self-serve button, which is a friction point. An actionable improvement would be to publish a *model card* for Cursor's in-house models (Tab, Apply, Composer) describing training data composition, evaluation results, and known failure modes, plus a per-account data-deletion UI directly in account settings. **Openness spectrum.** Cursor sits on the *closed, proprietary* end — the editor itself is a closed-source fork of the open VS Code, the in-house models are closed-weights, and there is no published model or system card. This places safety reasoning largely outside external auditability: independent researchers cannot replicate evaluations or examine training data, so accountability for model behavior reduces to whatever Anysphere chooses to disclose, with the public Trust Center being the primary verification surface.

6.c

Fairness (defined as "equal quality of code suggestions across programming languages, framework ecosystems, and natural human languages used in comments"): Cursor publishes no language-by-language quality benchmarks or disparity audits, so the only signals are anecdotal forum reports. An improvement would be to publish a periodic evaluation comparing acceptance rate / correct-on-first-try rate across, say, the top 20 languages and a bundle of low-resource languages, with the same dataset and versioning. **Transparency** (defined as "a published model card with training-data sources, evaluation results, and known failure modes"): no such document exists for Cursor's in-house Tab, Apply, or Composer models. The Trust Center fills part of this gap on the *operational* side (subprocessors, SOC 2) but leaves the *model* side unaddressed; publishing model cards in the Hugging Face / Anthropic format would close it. **Accountability** (defined as "a documented mechanism by which a user can contest or appeal a model output and receive a human response"): the only available channels are the community forum and a generic privacy email; there is no labelled appeals process or response-time SLA. An improvement would be a "Report this suggestion" button in-editor that routes into a ticket queue with a published median-response-time metric on the Trust Center, mirroring how mature platforms (e.g. Cloudflare's abuse pipeline) make accountability legible.

6.d

Accountability tools currently available to users. (1) A per-account Privacy Mode toggle that opts out of training use and enforces ZDR with model providers. (2) Admin-enforced Privacy Mode for Team / Enterprise members. (3) A public subprocessor list and Trust Center (`trust.cursor.com`). (4) A community forum for raising concerns publicly. (5) A privacy contact (`privacy@anysphere.inc`) for access, correction, and deletion requests under the Privacy Policy. Notably absent: a self-serve data-export or one-click account-deletion UI, an in-app appeals mechanism, and a public incident dashboard.

Proposed addition: a self-serve "Data Export & Delete" page in account settings that lets a user (a) download every prompt, suggestion, and telemetry event Anysphere has retained for their account in a portable JSON, and (b) trigger a verified deletion with a confirmation receipt. This makes the rights already promised in the Privacy Policy operationally legible rather than gated behind an email request, and gives users a concrete check on whether the company actually retained what its policy says it retained.

6.e

Cursor itself has not (as of public reporting in late 2025) been named as a defendant in a copyright-or-training-data lawsuit, but it inherits exposure from the foundation models it routes to: the closest analogues are the GitHub Copilot class action (*Doe v. GitHub*) and the 2025 *Bartz v. Anthropic* settlement ($1.5B) which confirmed that pirated books were among Anthropic's training data — and Anthropic models are one of Cursor's primary providers. Anysphere has not published a model card or training-data disclosure for its in-house Tab / Apply / Composer models, so the upstream provenance for those fine-tunes is opaque, and the foundation-model providers it relies on similarly do not publish full training-set inventories. In the Terms of Service, Anysphere explicitly assigns generated suggestions back to the user ("Anysphere hereby assigns to you all of our right, title, and interest if any in and to any Suggestions") and disclaims warranties including "non-infringement," meaning a Free / Pro user who unwittingly accepts a suggestion that turns out to copy copyrighted code carries the IP risk personally; Enterprise customers receive an explicit IP indemnity in the MSA. In my own use I did not observe verbatim **memorization** (e.g. identifiable long-form copyrighted comments), but I have seen Cursor produce near-verbatim implementations of small, well-known utility functions and idioms (*de facto* **extraction**) where the suggestion appears to reconstruct an idiomatic form learned from training rather than synthesize it freshly — usually innocuous for boilerplate, but in principle the same mechanism could surface non-trivially copyrighted code without provenance. The combined picture is that Cursor's copyright exposure is largely externalized: upstream providers carry training-data risk, downstream individual users carry indemnity risk, and the product layer in between makes no first-class disclosures about either.